

IT 7075C — Applied AI for Cybersecurity

Mini Project: Configure AI Tools and Environments

Name: Dharmin Patel

Repository: <https://github.com/patel5d2/ai-tools-setup>

Video demonstration (§3.9): <PASTE UNLISTED YOUTUBE / DRIVE LINK HERE>

Platform: MacBook Pro (Mac14,10), Apple M2 Pro, 12 cores (8P/4E), 16 GB unified memory, macOS 26.5.1 (arm64)

Editor: Visual Studio Code — Insiders

§3.6 (Jetstream2) is not included: the competency was announced as unavailable, so its 10 points redistribute across the remaining competencies.

§3.1 — Integrated development environment (8 pts)

VS Code Insiders is installed at `/Applications/Visual Studio Code - Insiders.app`, with the project folder `ai-tools-setup/` opened as the workspace root.

Required extensions, verified installed:

Extension	Identifier	Version
Python	<code>ms-python.python</code>	2026.4.0
Pylance	<code>ms-python.vscode-pylance</code>	2026.3.1
Python Debugger	<code>ms-python.debugpy</code>	2026.6.0
Jupyter	<code>ms-toolsai.jupyter</code>	2026.6.2026071501
GitHub Pull Requests	<code>github.vscode-pull-request-github</code>	0.166.0

Listed with the Insiders CLI:

```
$ "/Applications/Visual Studio Code - Insiders.app/Contents/Resources/app/bin/code" \
  --list-extensions --show-versions | grep -E 'ms-python|ms-toolsai|github'
```

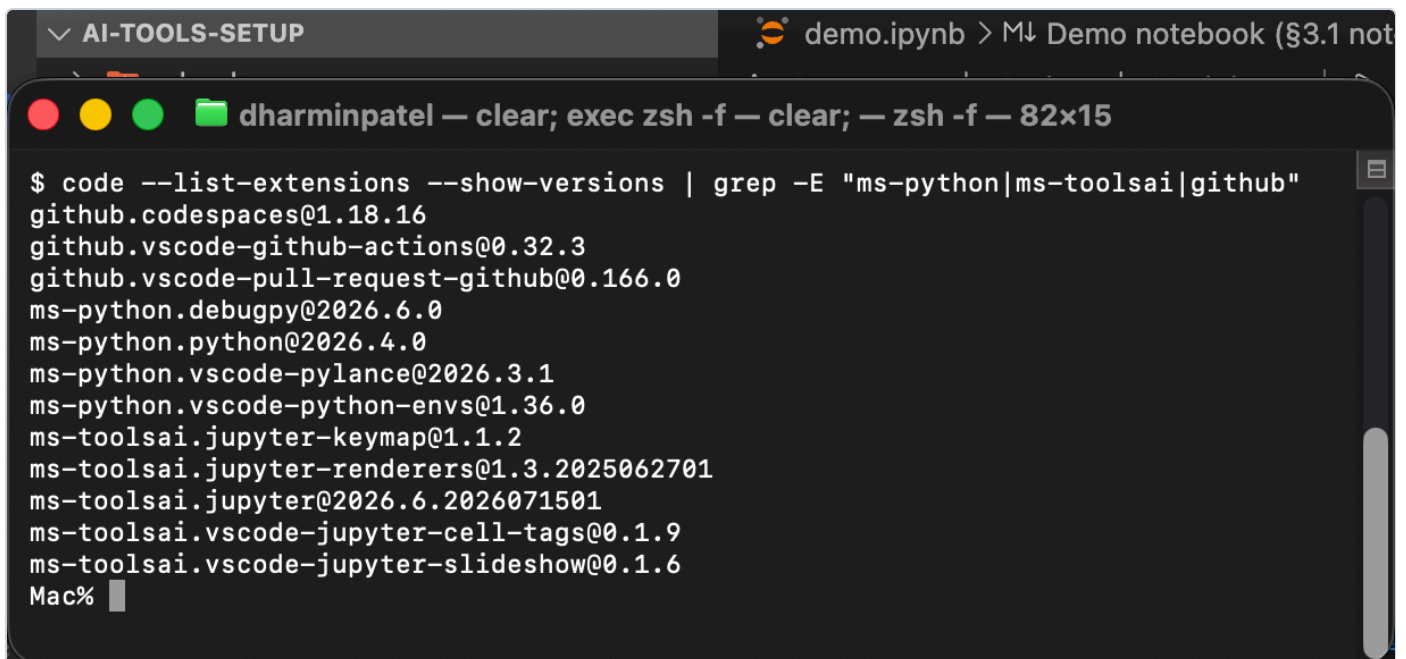
Script executed in the editor — `hello.py` :

```
$ python hello.py
Hello from the isolated environment.
Interpreter: /Users/dharminpatel/IT 7075C - APPLIED AI FOR CYBERSECURITY/ai-tools-
setup/.venv/bin/python
```

Notebook cell executed in the editor — `demo.ipynb`, first cell:

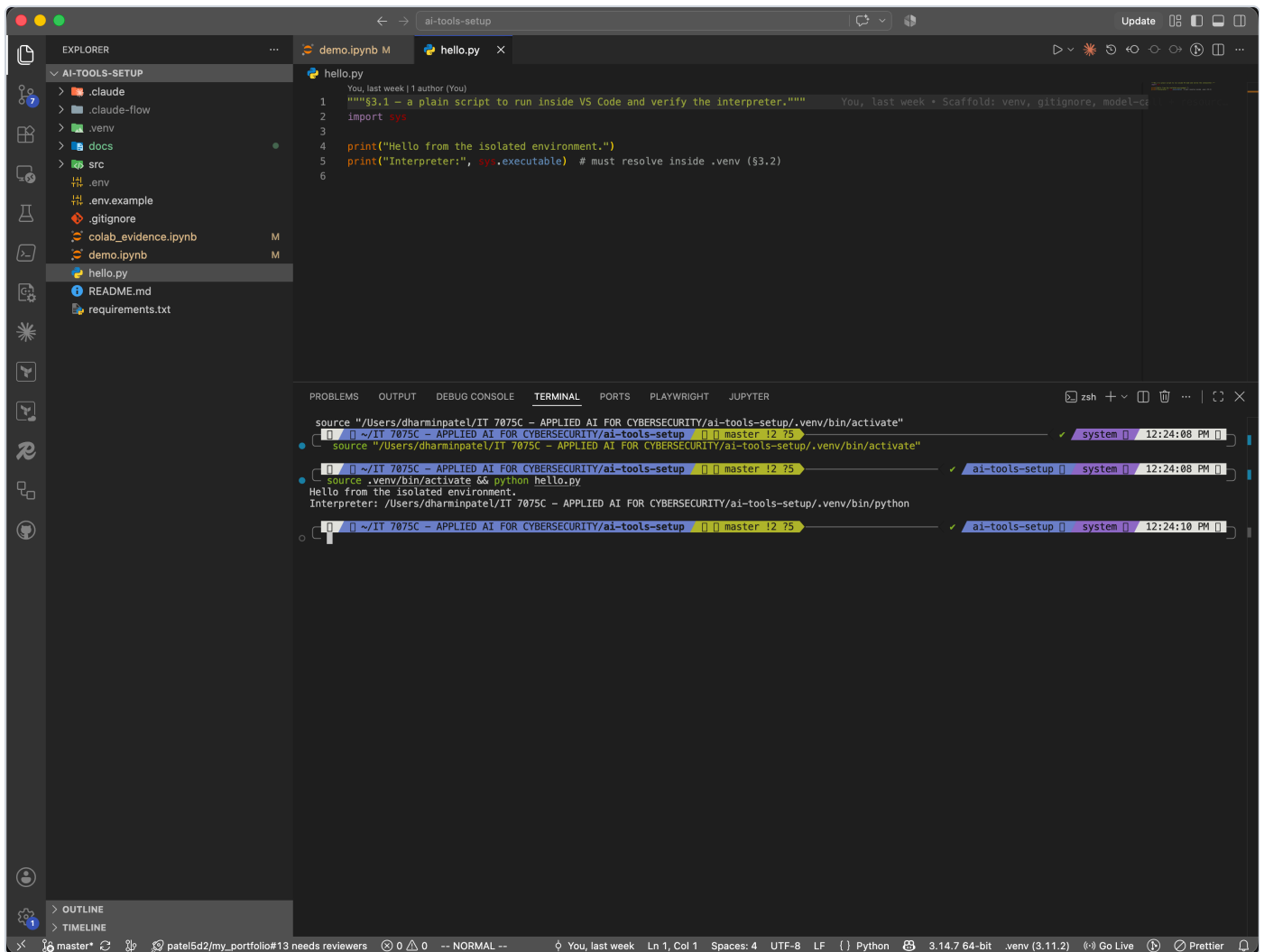
```
Kernel interpreter: /Users/dharminpatel/IT 7075C - APPLIED AI FOR CYBERSECURITY/ai-tools-
setup/.venv/bin/python
```

Both print the interpreter path, which is the same evidence §3.2 requires: the editor is running code inside the project virtual environment, not the system Python.

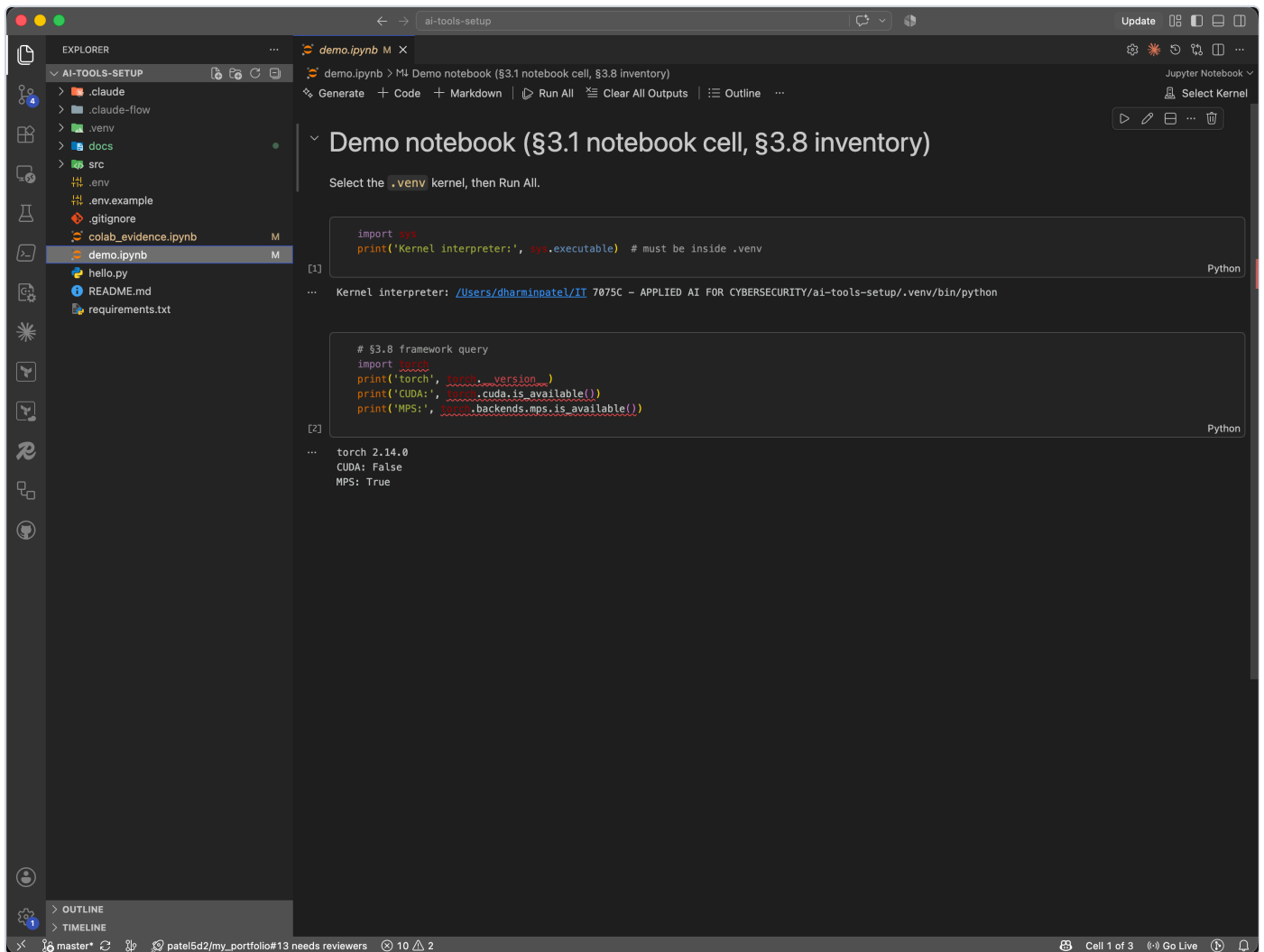


```
AI-TOOLS-SETUP demo.ipynb > M Demo notebook (§3.1 not
dharmipatel — clear; exec zsh -f — clear; — zsh -f — 82x15
$ code --list-extensions --show-versions | grep -E "ms-python|ms-toolsai|github"
github.codespaces@1.18.16
github.vscode-github-actions@0.32.3
github.vscode-pull-request-github@0.166.0
ms-python.debugpy@2026.6.0
ms-python.python@2026.4.0
ms-python.vscode-pylance@2026.3.1
ms-python.vscode-python-envs@1.36.0
ms-toolsai.jupyter-keymap@1.1.2
ms-toolsai.jupyter-renderers@1.3.2025062701
ms-toolsai.jupyter@2026.6.2026071501
ms-toolsai.vscode-jupyter-cell-tags@0.1.9
ms-toolsai.vscode-jupyter-slideshow@0.1.6
Mac%
```

§3.1 Installed extensions listed with the Insiders CLI — Python, Pylance, Python Debugger, Jupyter and GitHub Pull Requests, with versions



§3.1 Explorer showing the open project directory, and the integrated terminal running hello.py



§3.1 demo.ipynb with an executed cell and its output

§3.2 — Isolated Python environments (10 pts)

The environment is a standard `venv` at `.venv/` in the project root, built on the Python 3.11 framework build rather than the Homebrew 3.14 interpreter — PyTorch does not yet publish wheels for 3.14, and `torch` is required by §3.8.

Commands issued:

```
$ /Library/Frameworks/Python.framework/Versions/3.11/bin/python3.11 -m venv .venv # create
$ source .venv/bin/activate # activate
$ python -m pip install -r requirements.txt # install
deps
$ deactivate #
deactivate
```

Verification — the interpreter resolves inside the environment, not the system install:

```
$ python -V
Python 3.11.2

$ which python
/Users/dharminpatel/IT 7075C - APPLIED AI FOR CYBERSECURITY/ai-tools-setup/.venv/bin/python
```

Contrast with the deactivated shell, where `which python3` returns `/opt/homebrew/bin/python3` (Python 3.14.7). The prefix difference is the proof of isolation.

`requirements.txt` :

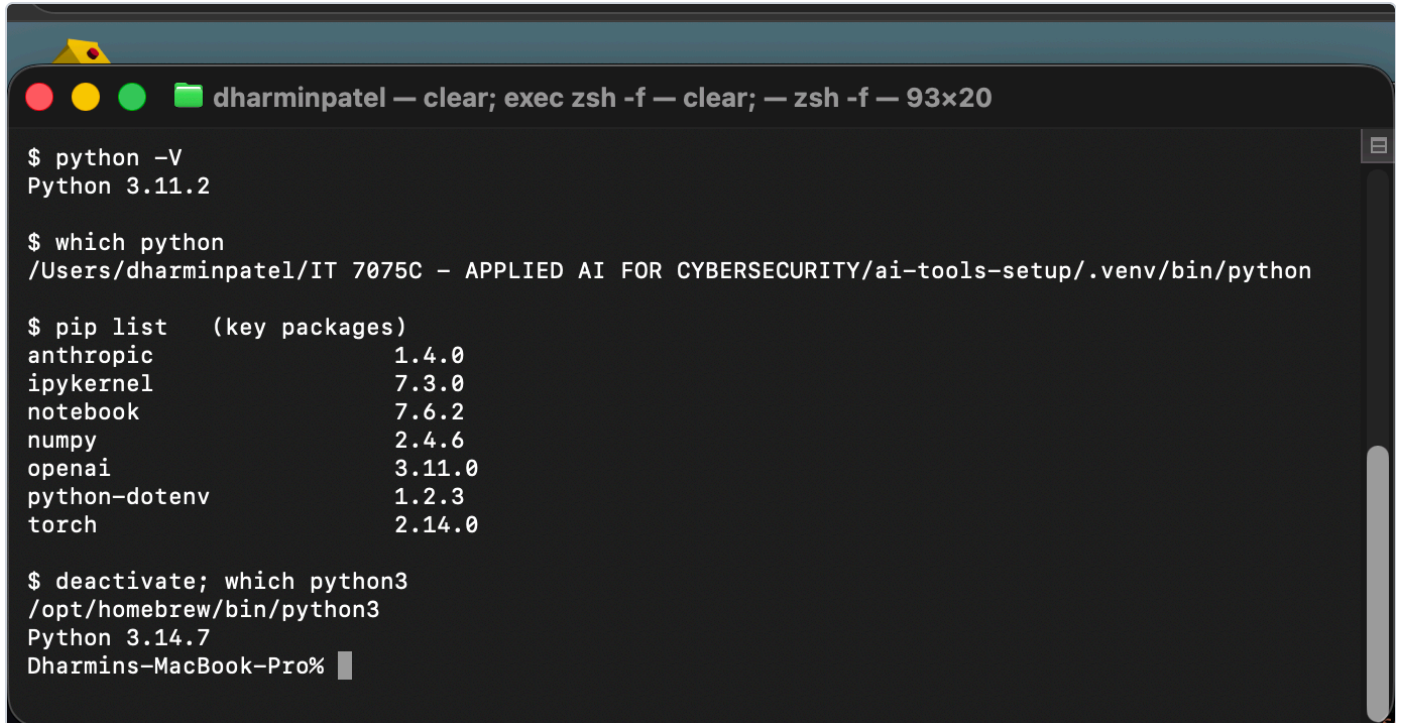
```
python-dotenv
numpy
anthropic
openai
torch
ipykernel
notebook
```

`pip list` (abridged — key packages):

```
anthropic      1.4.0
ipykernel      7.3.0
notebook       7.6.2
numpy          2.4.6
openai         3.11.0
python-dotenv  1.2.3
torch          2.14.0
```

The environment was registered as a named Jupyter kernel so it is selectable in the notebook kernel picker as well as the interpreter picker:

```
$ python -m ipykernel install --user --name ai-tools-setup \
    --display-name "Python (.venv ai-tools-setup)"
Installed kernelspec ai-tools-setup in /Users/dharminpatel/Library/Jupyter/kernels/ai-tools-
setup
```



A terminal window titled "dharminpatel — clear; exec zsh -f — clear; — zsh -f — 93x20". The terminal shows the following commands and output:

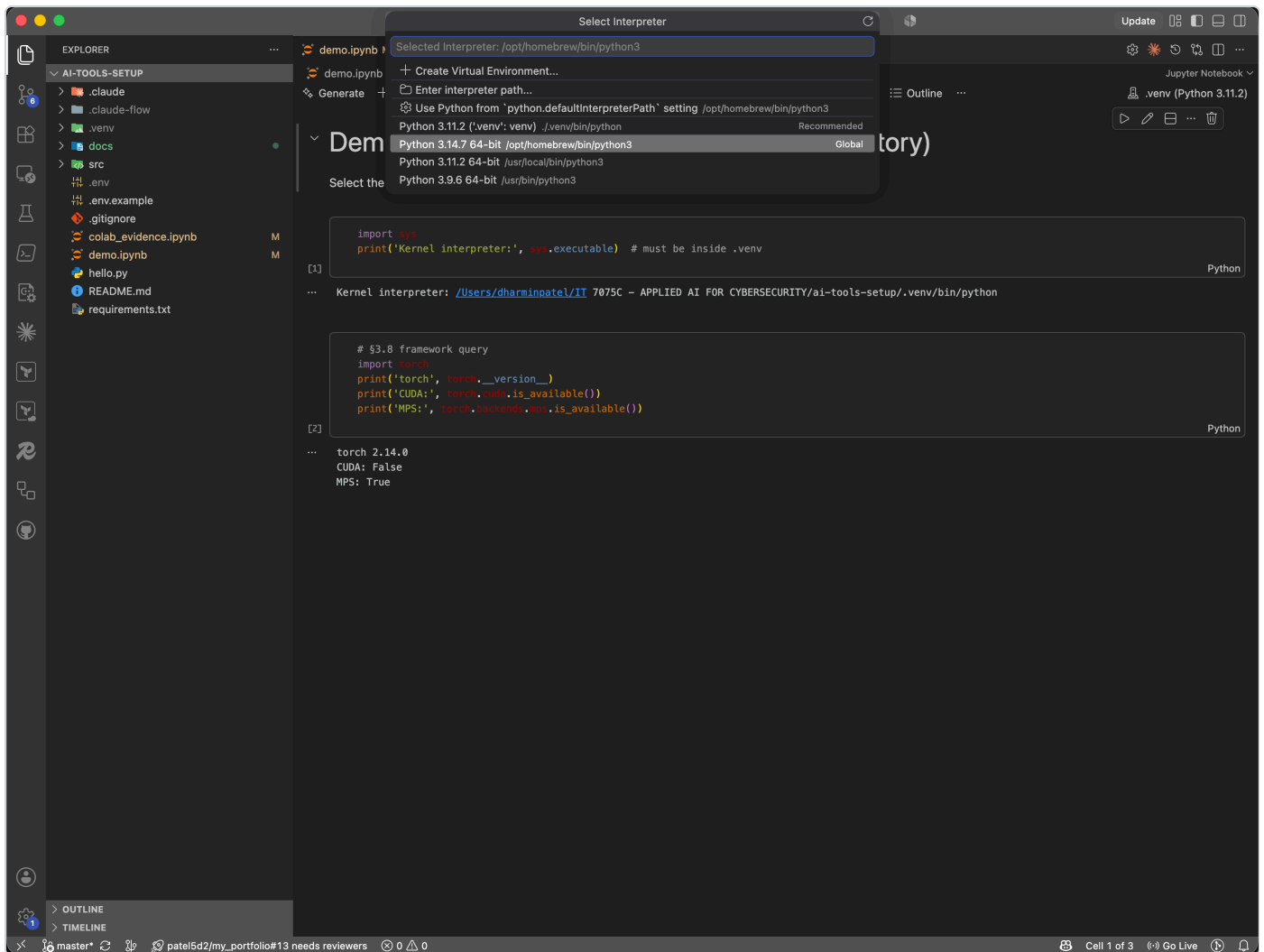
```
$ python -V
Python 3.11.2

$ which python
/Users/dharminpatel/IT 7075C - APPLIED AI FOR CYBERSECURITY/ai-tools-setup/.venv/bin/python

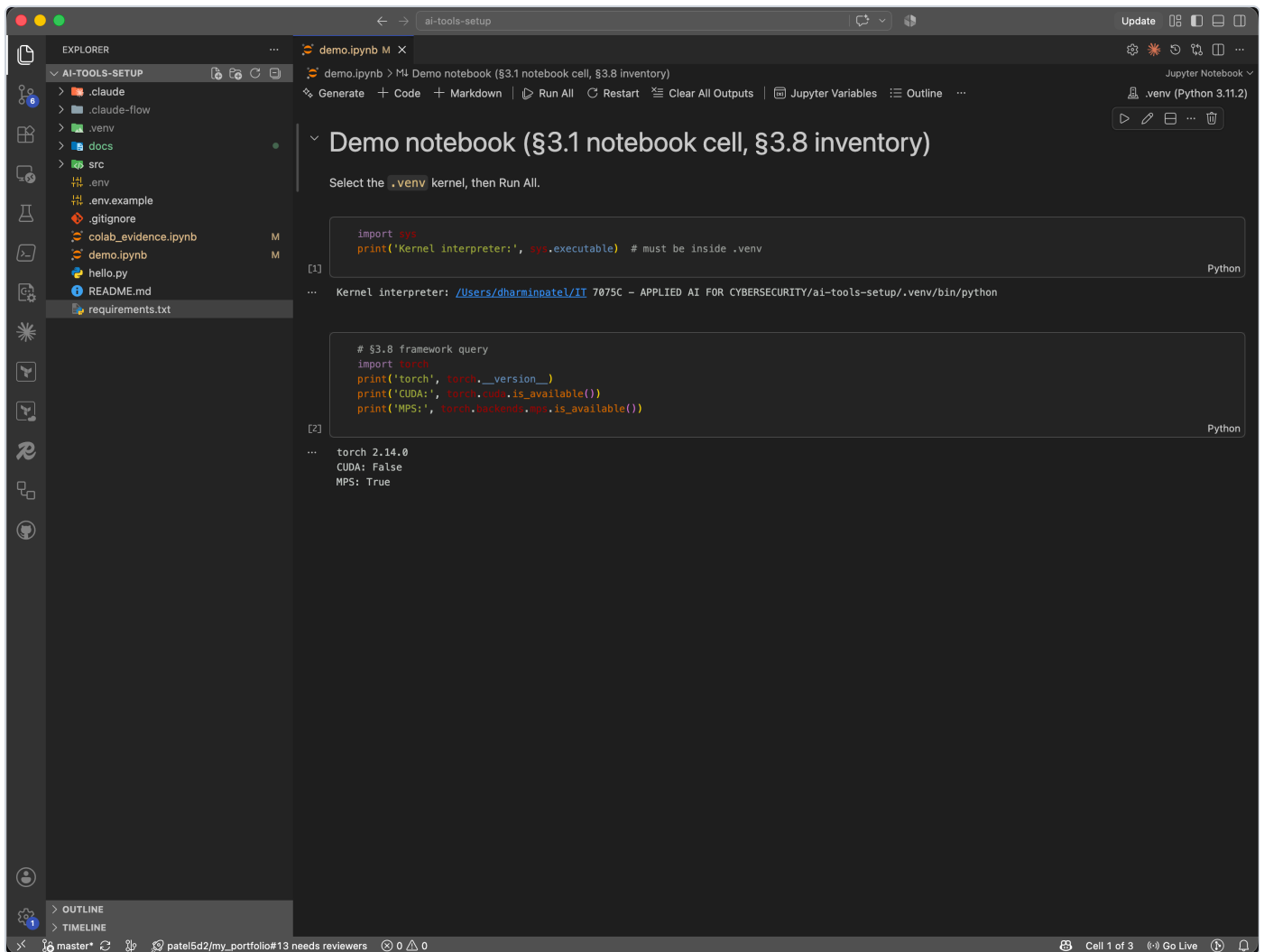
$ pip list    (key packages)
anthropic      1.4.0
ipykernel      7.3.0
notebook       7.6.2
numpy          2.4.6
openai         3.11.0
python-dotenv  1.2.3
torch          2.14.0

$ deactivate; which python3
/opt/homebrew/bin/python3
Python 3.14.7
Dharmin-MacBook-Pro%
```

§3.2 Terminal showing python -V, which python, and pip list with the venv active



\$3.2 VS Code interpreter picker with the .venv path selected



§3.2 Notebook kernel picker with the .venv kernel selected

§3.3 – Version control and remote repositories (10 pts)

Repository: <https://github.com/patel5d2/ai-tools-setup> (public)

Local identity:

```
$ git config --global user.name
Dhrm976
$ git config --global user.email
dharminp976@gmail.com
```

Commit history (more than the two required substantive commits):

```
$ git log --oneline
95f9d42 Rebuild venv at current path; pin model to claude-opus-5; add Colab evidence notebook
3629370 Remove outdated .DS_Store file
6c760c1 Update demo notebook: adjust execution counts and add torchvision version check
...
6432a49 Add numpy – torch needs it and it silences the MPS-init warning
8dd37de Scaffold: venv, gitignore, model-call + resource-inventory scripts
```

Credential and environment exclusion — `.gitignore`:

```
# Secrets – never commit (§3.3, §3.5)
.env

# Virtual environment (§3.2, §3.3)
.venv/

# Python
__pycache__/
*.pyc
.ipynb_checkpoints/

# macOS / tooling noise
.DS_Store
.claude-flow/
```

Proof the exclusion works. A real `.env` containing a live API key exists in the working directory, yet `git status` is clean and does not list it:

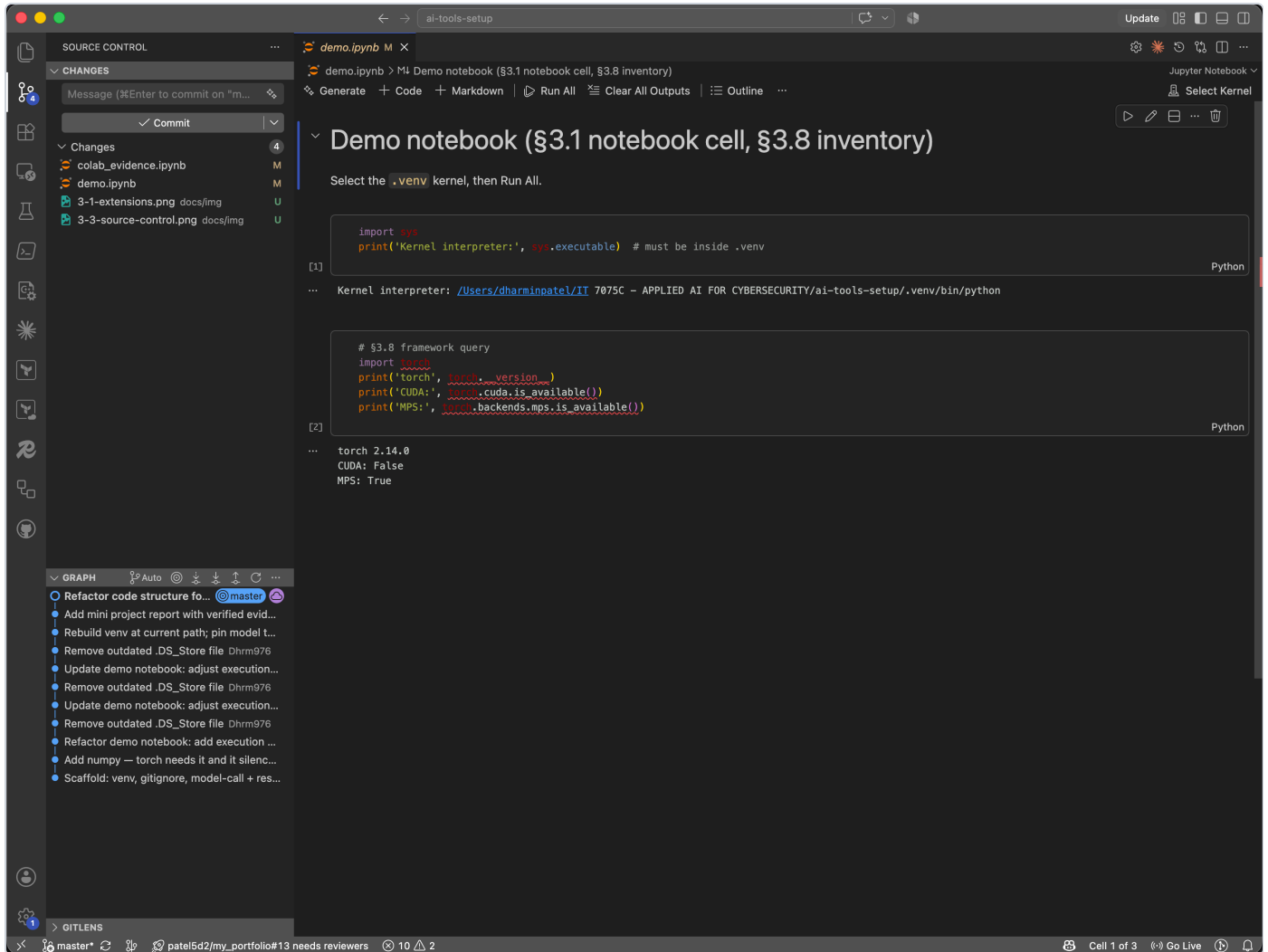
```
$ ls -l .env
-rw-r--r-- 1 dharminpatel 127 Sep  4 13:46 .env

$ git status --short
(no output – working tree clean)

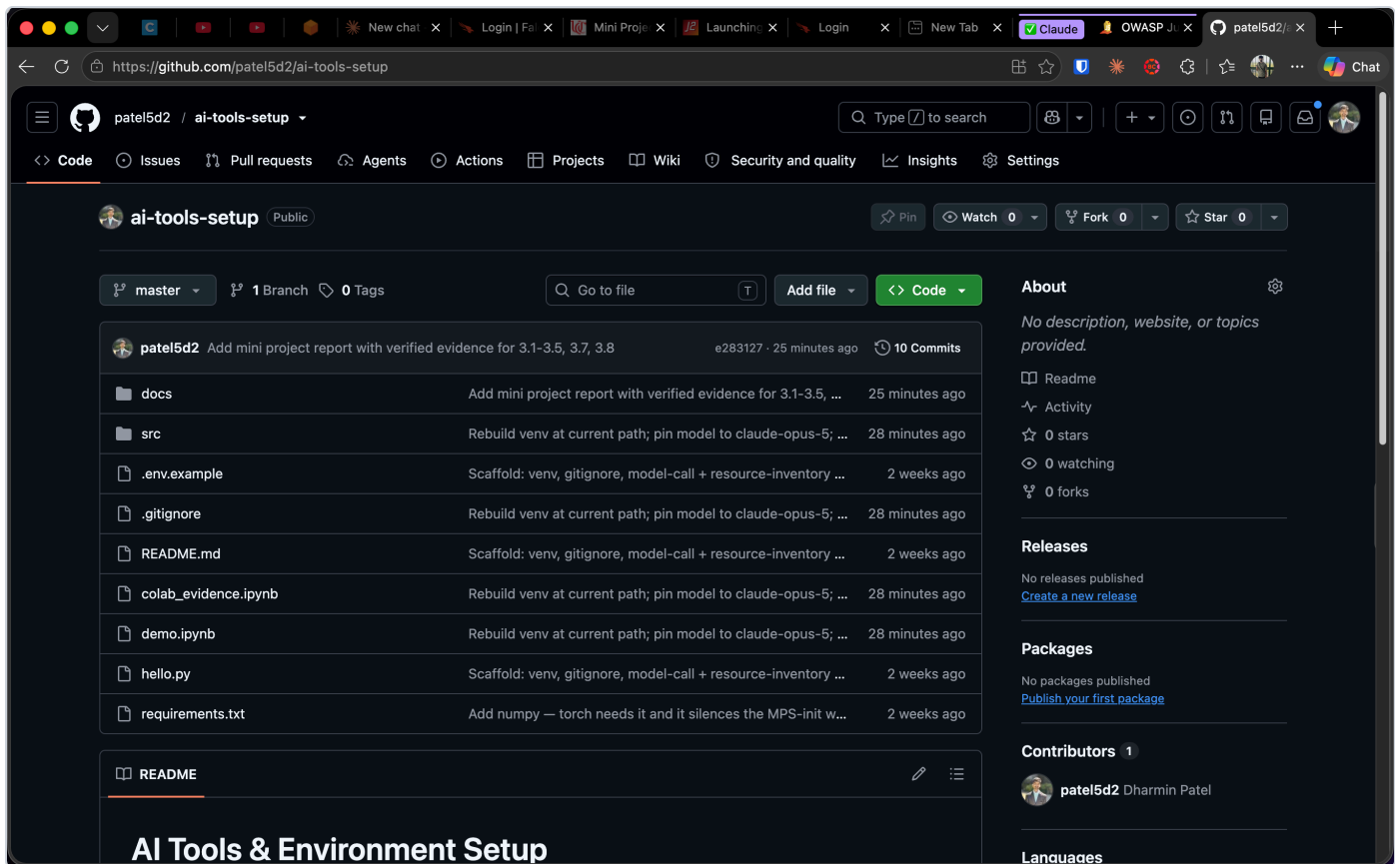
$ git ls-files | grep -c '^\.env$'
0
```

`.env.example` is committed, carrying the variable names but placeholder values, so the repository documents what is required without disclosing anything:

```
# Copy to .env and fill in. .env is gitignored – never commit real keys (§3.5).
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
```



§3.3 VS Code Source Control panel showing the origin remote and commit history



§3.3 GitHub repository page

```
dharmminpatel — clear; exec zsh -f — clear; — zsh -f — 80x13

$ ls -l .env # a real key file exists on disk
-rw-r--r--@ 1 dharmminpatel staff 127 Sep  4 13:46 .env

$ git check-ignore -v .env # which rule excludes it
.gitignore:2:.env .env

$ git ls-files | grep env # what git actually tracks
.env.example

$ git status --short # .env never appears
?? docs/img/
Dharmmin-MacBook-Pro%
```

§3.3 Terminal: ls -l .env alongside a clean git status

§3.4 — Hosted notebooks and persistent storage (10 pts)

Notebook: `colab_evidence.ipynb`, executed in Google Colab.

Google Drive is mounted at `/content/drive`, a file is written beneath `MyDrive`, the runtime is then restarted, and the file is read back after the runtime reconnects:

```
from google.colab import drive
drive.mount('/content/drive')

from pathlib import Path
import datetime
p = Path('/content/drive/MyDrive/IT7075C/persistence.txt')
p.parent.mkdir(parents=True, exist_ok=True)
p.write_text(f'written before restart at {datetime.datetime.now()}\n')
```

After **Runtime ▶ Restart session**, the runtime is a fresh virtual machine — `/content` is empty — but Drive is durable storage outside it, so remounting recovers the file:

```
drive.mount('/content/drive')
p = Path('/content/drive/MyDrive/IT7075C/persistence.txt')
print('survived restart:', p.exists())
print(p.read_text())
```

This is the point of the competency: Colab's local filesystem is ephemeral and disappears with the runtime, so anything that must outlive a session has to be written to mounted Drive.

§3.4 Drive mount confirmation output

§3.4 Drive mount confirmation output

The screenshot shows a Google Colab notebook titled 'colab_evidence.ipynb'. The interface includes a top menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. On the left, a 'Table of contents' sidebar lists sections: 'IT 7075C — Colab evidence...', '§3.4 — Mount Drive and ...', '— Restart the runtime ...', '§3.5 — Model call with th...', and '§3.8 — Resource inventor...'. The main area displays code cells. Cell [10] shows code to create a file in Google Drive and write text to it. Cell [11] shows code to mount the drive and read the file back after a restart. Cell [12] shows code to install and use the Anthropic API. Cell [13] shows code to create a message and call the Anthropic model. The output of cell [11] shows the file contents read back after a restart.

```

[10] ✓ 0s
4 p = Path('/content/drive/MyDrive/IT7075C/persistence.txt')
5 p.parent.mkdir(parents=True, exist_ok=True)
6 p.write_text(f'written before restart at {datetime.datetime.now()}\n')
7 print('wrote:', p)
8 print(p.read_text())

wrote: /content/drive/MyDrive/IT7075C/persistence.txt
written before restart at 2026-09-10 16:08:33.771674

← Restart the runtime now (Runtime • Restart session), then continue.

[11] ✓ 0s
1 # After the restart the runtime is a fresh VM: /content is empty but Drive
  persists.
2 from google.colab import drive
3 drive.mount('/content/drive')
4
5 from pathlib import Path
6 p = Path('/content/drive/MyDrive/IT7075C/persistence.txt')
7 print('survived restart:', p.exists())
8 print(p.read_text())

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
survived restart: True
written before restart at 2026-09-10 16:08:33.771674

§3.5 — Model call with the key from Colab Secrets

[12] ✓ 10s
1 from google.colab import userdata
2 import os
3
4 os.environ['ANTHROPIC_API_KEY'] = userdata.get('ANTHROPIC_API_KEY') # key
  never appears in the notebook
5 !pip install -q anthropic

1.3/1.3 MB 21.3 MB/s eta 0:00:00

[13] ✓ 9s
1 from anthropic import Anthropic
2
3 client = Anthropic() # reads ANTHROPIC_API_KEY from the environment
4 msg = client.messages.create(
5     model='claude-opus-5', max_tokens=100,
6     messages=[{'role': 'user', 'content': 'Reply with one short sentence
  confirming the API call worked.'}],
7 )
  
```

§3.4 File written to Drive, with the file visible in the Colab file browser

§3.4 Post-restart cell showing survived restart: True and the file contents read back

§3.4 Post-restart cell showing survived restart: True and the file contents read back

§3.5 — Programmatic model access with managed credentials (12 pts)

Provider: **Anthropic**, model `claude-opus-5`.

The same source pattern works in both environments because both supply the key through the process environment — only the *source* of that environment variable differs.

Credential-loading code (`src/model_test.py`):

```

import os
from dotenv import load_dotenv

load_dotenv()          # local: reads .env. In Colab this is a no-op.
                        # Colab: the key is injected into os.environ from Secrets instead.

from anthropic import Anthropic
client = Anthropic()   # reads ANTHROPIC_API_KEY from the environment
model = os.getenv("MODEL", "claude-opus-5")
msg = client.messages.create(
    model=model, max_tokens=100,
    messages=[{"role": "user", "content": PROMPT}],
)
print(f"[Anthropic {model}] {msg.content[0].text}")

```

No key is ever a literal in the source. `Anthropic()` is constructed with no arguments and the SDK resolves `ANTHROPIC_API_KEY` from the environment itself.

Local environment — key from `.env` (gitignored):

```

$ python src/model_test.py
[Anthropic claude-opus-5] API call successful – everything's working as expected.

```

Colab environment — key from the Secrets panel:

```

from google.colab import userdata
import os
os.environ['ANTHROPIC_API_KEY'] = userdata.get('ANTHROPIC_API_KEY')

```

The key is stored in Colab's  **Secrets** sidebar with *Notebook* access enabled. It is held against the Google account, not inside the notebook, so the notebook can be shared or committed without leaking the credential — the Colab analogue of `.env` plus `.gitignore`.

All screenshots below are cropped or redacted so no key material is visible. The Secrets panel is captured with the value hidden (the eye toggle left off).

```
dharminpatel — clear; exec zsh -f — clear; — zsh -f — 85x4

$ python src/model_test.py
[Anthropic claude-opus-5] API call successful — everything's working as expected.
Dharmins-MacBook-Pro%
```

§3.5 Local model response in the VS Code terminal

The screenshot displays a Google Colab notebook titled 'colab_evidence.ipynb'. On the left, the 'Secrets' panel is open, showing a secret named 'ANTHROPIC_API_KEY' with its value hidden. Below the secrets, there is a section for 'Access your secret keys in Python via:' with a code snippet: `from google.colab import userdata; userdata.get('secretName')`. The main notebook area shows two code cells. The first cell (12) imports 'userdata' from 'google.colab' and sets the 'ANTHROPIC_API_KEY' environment variable. The second cell (13) imports 'Anthropic' and sends a message to the 'claude-opus-5' model, which responds with 'API call successful — everything's working as expected.' Below the code cells, a section titled '§3.8 — Resource inventory on the Colab runtime' is shown, containing a code cell (14) that runs system commands like 'nvidia-smi', 'lscpu', and 'free -h'. The output of these commands is displayed, showing system details such as 'no NVIDIA GPU attached', 'CPU op-mode(s): 32-bit, 64-bit', and 'CPU family: GenuineIntel'.

§3.5 Colab Secrets panel with ANTHROPIC_API_KEY present, value hidden, notebook access on

The screenshot shows a Google Colab notebook titled 'colab_evidence.ipynb'. The interface includes a top menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. A left sidebar contains a 'Table of contents' with links to sections like '§3.4 — Mount Drive and ...', '§3.5 — Model call with th...', and '§3.8 — Resource inventory on the Colab runtime'. The main area displays two code cells. The first cell, labeled '[12] ✓ 10s', contains Python code to import 'userdata' from 'google.colab', set environment variables for 'ANTHROPIC_API_KEY', and install 'anthropic'. A progress bar shows '1.3/1.3 MB 21.3 MB/s eta 0:00:00'. The second cell, labeled '[13] ✓ 9s', contains Python code to import 'Anthropic', create a client, and send a message to the model. Below the code, it says 'API call successful — everything's working as expected.' The third cell, labeled '[14] ✓ 0s', contains shell commands to check for an NVIDIA GPU and system resources. The output shows that no GPU is attached and lists system details like architecture (x86_64), CPU (Intel Xeon), and memory (79 MB). The bottom status bar shows 'Variables', 'Terminal', '12:08 PM', and 'Python 3'.

```
[12] ✓ 10s
1 from google.colab import userdata
2 import os
3
4 os.environ['ANTHROPIC_API_KEY'] = userdata.get('ANTHROPIC_API_KEY') # key
  never appears in the notebook
5 !pip install -q anthropic
  1.3/1.3 MB 21.3 MB/s eta 0:00:00

[13] ✓ 9s
1 from anthropic import Anthropic
2
3 client = Anthropic() # reads ANTHROPIC_API_KEY from the environment
4 msg = client.messages.create(
5     model='claude-opus-5', max_tokens=100,
6     messages=[{'role': 'user', 'content': 'Reply with one short sentence
  confirming the API call worked.'}],
7 )
8 print(msg.content[0].text)

API call successful — everything's working as expected.

§3.8 — Resource inventory on the Colab runtime

[14] ✓ 0s
1 # System utility. If the runtime is CPU-only, nvidia-smi is absent — that is a
  valid result.
2 !nvidia-smi || echo 'no NVIDIA GPU attached (Runtime > Change runtime type to
  request one)'
3 !lscpu | head -20
4 !free -h

/bin/bash: line 1: nvidia-smi: command not found
no NVIDIA GPU attached (Runtime > Change runtime type to request one)
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               46 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      2
On-line CPU(s) list:         0,1
Vendor ID:                   GenuineIntel
Model name:                   Intel(R) Xeon(R) CPU @ 2.20GHz
CPU family:                   6
Model:                       79
Thread(s) per core:          2
Core(s) per socket:          1
Socket(s):                    1
Stepping:                     0
BogoMIPS:                     4399.99
```

§3.5 Colab cell showing the model response

§3.6 — Cloud virtual machine deployment on Jetstream2

Not attempted — announced as unavailable. Under the stated contingency the ten points are redistributed across the remaining competencies.

§3.7 — Virtualization and isolated targets (12 pts)

Platform: Docker Desktop. Target: OWASP Juice Shop.

Why a container rather than Metasploitable2. This machine is Apple Silicon (arm64). VirtualBox has no arm64 host build, and Metasploitable2 ships only as a 32-bit x86 image, so it cannot be virtualized here without full CPU emulation. The assignment's stated alternative for exactly this case is a containerized target, so OWASP Juice Shop was deployed instead.

Network configuration — user-defined bridge, NAT, loopback-bound publish:

```
$ docker network inspect pentest-lab \
  --format '{{.Driver}} subnet={{range .IPAM.Config}}{{.Subnet}} gw={{.Gateway}}'
{{end}}'
driver=bridge subnet=172.28.0.0/16 gw=172.28.0.1

$ docker run -d --name juice-shop --network pentest-lab \
  -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

Justification of the network mode. A user-defined bridge network in NAT mode was chosen over host networking for three reasons:

1. **The target is deliberately vulnerable.** On a bridge it has no address on the physical LAN, so nothing outside this machine can route to it. Host networking would place it directly on the host's interfaces and expose it to the local network.
2. **The publish is bound to 127.0.0.1, not 0.0.0.0.** The default `-p 3000:3000` listens on every host interface, including Wi-Fi. Binding the loopback address explicitly means only this machine can reach the service — the smallest exposure that still permits host-to-guest testing.
3. **A user-defined bridge gives container-name DNS,** so lab hosts address each other by name (`juice-shop`) rather than by an address that changes on restart. The default bridge does not provide this.

Host-only networking was not selected because Docker Desktop on macOS has no equivalent mode; a loopback-bound bridge achieves the same containment goal.

Running guest:

```
$ docker ps
NAME          IMAGE                STATUS              PORTS
juice-shop    bkimminich/juice-shop Up 6 seconds        127.0.0.1:3000->3000/tcp
```

Reachability — service reached from the host:

```
$ nc -zv 127.0.0.1 3000
localhost [127.0.0.1] 3000 (hbc1) open

$ curl -sI http://127.0.0.1:3000
HTTP/1.1 200 OK
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Recruiting: /#/jobs

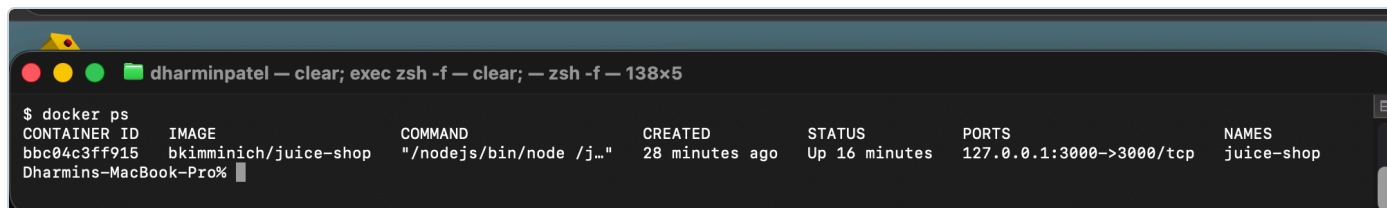
$ curl -s http://127.0.0.1:3000 | grep -io '<title>.*</title>'
<title>OWASP Juice Shop</title>
```

Reachability — ICMP on the lab network:

```
$ docker run --rm --network pentest-lab busybox ping -c 3 juice-shop
PING juice-shop (172.28.0.2): 56 data bytes
64 bytes from 172.28.0.2: seq=0 ttl=64 time=0.855 ms
64 bytes from 172.28.0.2: seq=1 ttl=64 time=0.089 ms
64 bytes from 172.28.0.2: seq=2 ttl=64 time=0.134 ms

--- juice-shop ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
```

An honest note on the ping. Pinging `172.28.0.2` *directly from macOS* fails with 100% packet loss, and this is expected rather than a misconfiguration — see the problem log below. Layer-3 reachability is therefore demonstrated on the lab network itself, and host-to-guest reachability is demonstrated at layer 4 and layer 7 through the published port, which is the path an attack tool would actually use against this target.



```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
bbc04c3ff915	bkimminich/juice-shop	"/nodejs/bin/node /j..."	28 minutes ago	Up 16 minutes	127.0.0.1:3000->3000/tcp	juice-shop

§3.7 docker ps showing the running guest and its port binding

```
dharminpatel — clear; exec zsh -f — clear; — zsh -f — 132x15

$ docker network inspect pentest-lab --format "driver={{.Driver}} subnet={{range .IPAM.Config}}{{.Subnet}} gw={{.Gateway}}{{end}}"
driver=bridge subnet=172.28.0.0/16 gw=172.28.0.1

$ docker network ls
NETWORK ID      NAME      DRIVER  SCOPE
05247a97335c    bridge   bridge  local
17c39f8fe370    host     host    local
1d251e286356    kind     bridge  local
67a24f4a8bae    none     null    local
f6b9108c0836    pentest-lab  bridge  local

$ docker inspect juice-shop --format "{{range $k,$v := .NetworkSettings.Networks}}{{$k}} -> {{$v.IPAddress}}{{end}}"
pentest-lab -> 172.28.0.2
Dharmins-MacBook-Pro%
```

§3.7 docker network inspect showing the bridge driver, subnet and gateway

```
dharminpatel — clear; exec zsh -f — clear; — zsh -f — 80x24

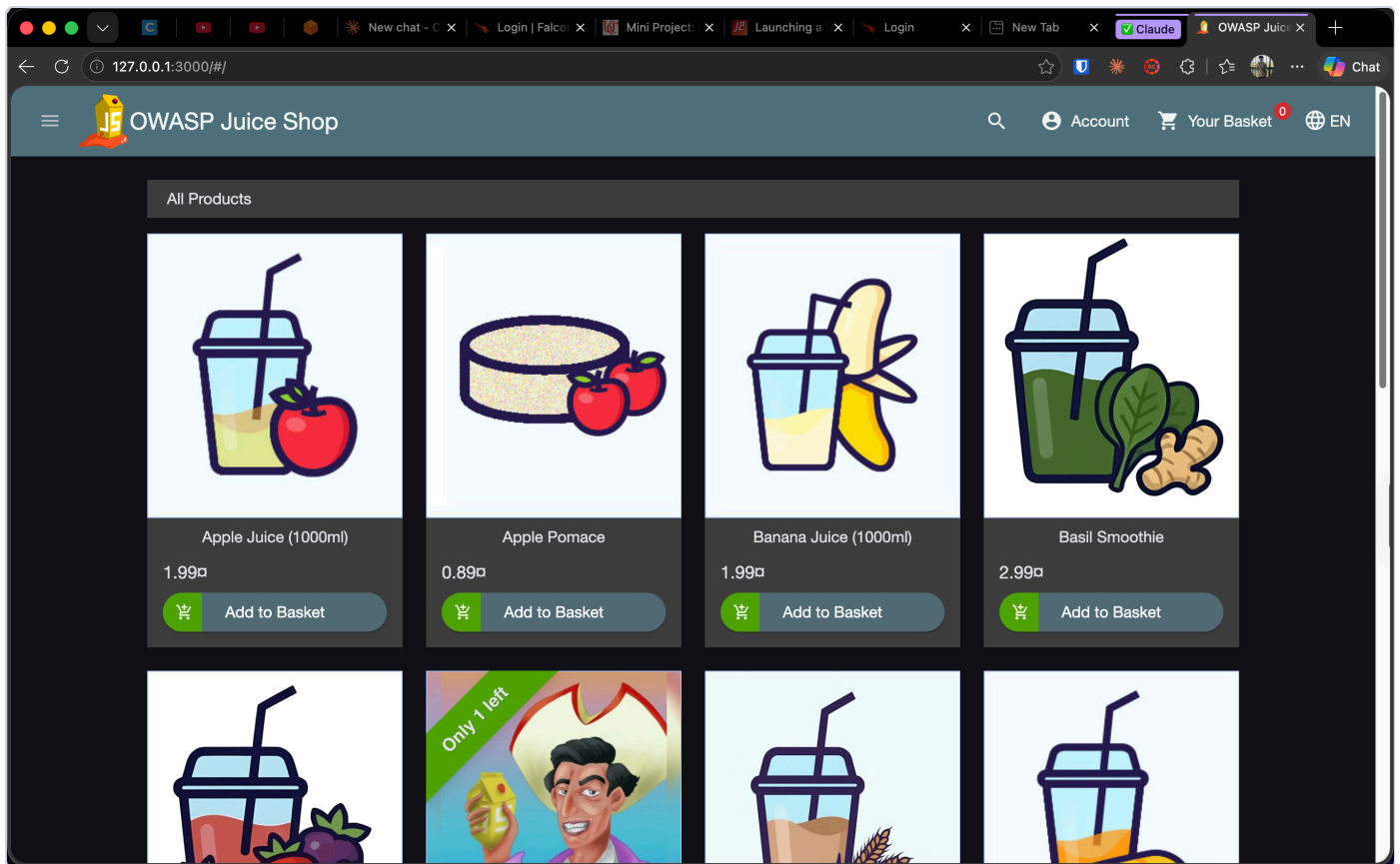
$ nc -zv 127.0.0.1 3000
localhost [127.0.0.1] 3000 (hbc) open

$ curl -sI http://127.0.0.1:3000
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
$ curl -s http://127.0.0.1:3000 | grep -io "<title>.*</title>"
<title>OWASP Juice Shop</title>

$ docker run --rm --network pentest-lab busybox ping -c 3 juice-shop
PING juice-shop (172.28.0.2): 56 data bytes
64 bytes from 172.28.0.2: seq=0 ttl=64 time=0.063 ms
64 bytes from 172.28.0.2: seq=1 ttl=64 time=0.086 ms
64 bytes from 172.28.0.2: seq=2 ttl=64 time=0.083 ms

--- juice-shop ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.063/0.077/0.086 ms
Dharmins-MacBook-Pro%
```

§3.7 nc, curl and ping output side by side



§3.7 Juice Shop landing page open in the host browser at 127.0.0.1:3000

§3.8 — Computational resource inventory (8 pts)

Collected by `src/resource_inventory.py`, which branches to the correct system utility for the platform it is running on and then runs the same framework query everywhere.

Local machine — MacBook Pro, Apple M2 Pro

System utility (`system_profiler SPHardwareDataType SPDDisplaysDataType` , serial numbers cropped):

```
Model Name: MacBook Pro
Model Identifier: Mac14,10
Chip: Apple M2 Pro
Total Number of Cores: 12 (8 Performance and 4 Efficiency)
Memory: 16 GB
```

```
Apple M2 Pro:
  Chipset Model: Apple M2 Pro
  Type: GPU
  Bus: Built-In
  Total Number of Cores: 19
  Metal Support: Metal 4
```

Framework query (`torch`):

```
CPUs (os.cpu_count): 12
torch 2.14.0
CUDA available: False
MPS (Apple GPU) available: True
```

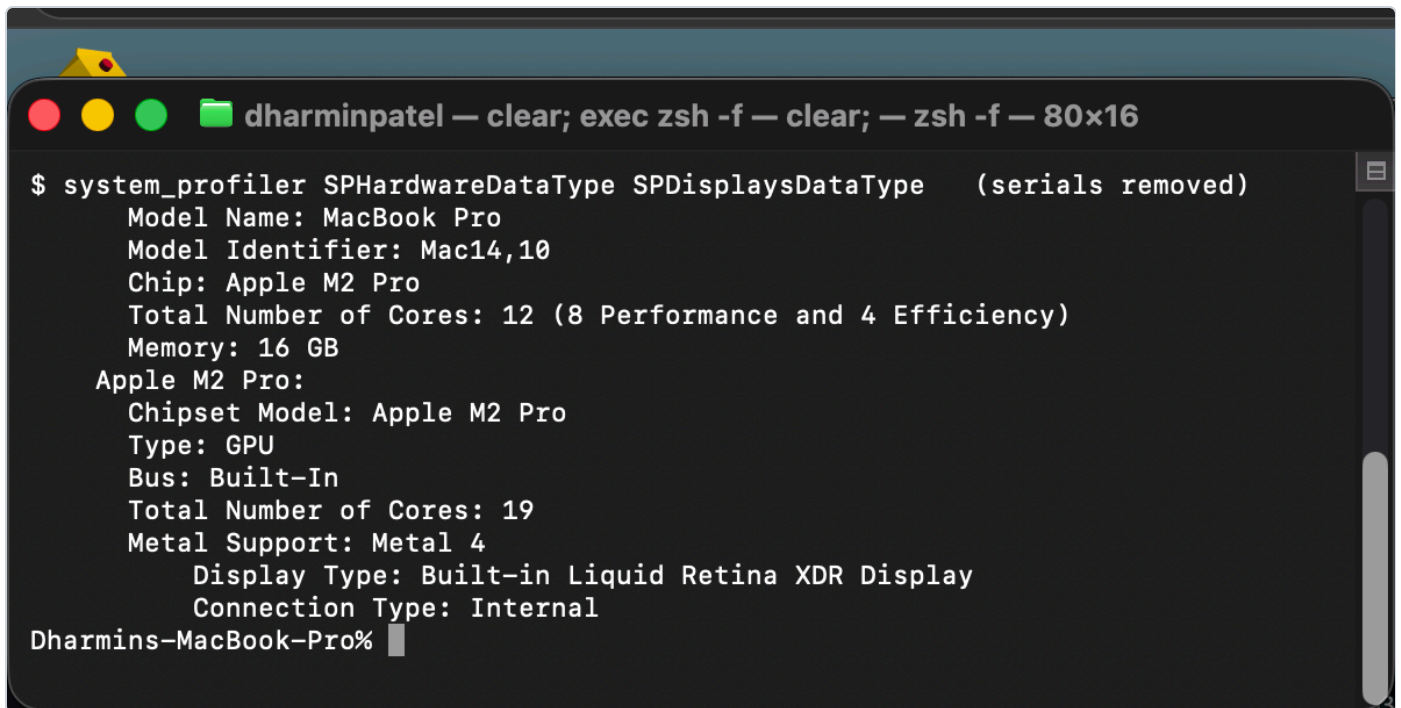
Interpretation. `CUDA available: False` is the correct and expected result: CUDA is NVIDIA's API, and this machine has an Apple integrated GPU, not an NVIDIA discrete card. The GPU is present and usable — through Metal, which PyTorch exposes as the MPS backend, reporting `True` . Reading only the CUDA line would wrongly conclude "no GPU"; the 19 GPU cores are available for acceleration, just through a different backend. Note also that memory is *unified*: the 16 GB is shared between CPU and GPU rather than the GPU holding separate dedicated VRAM.

Google Colab

System utility: `nvidia-smi` , plus `lscpu` and `free -h` .

Framework query: `torch.cuda.is_available()` .

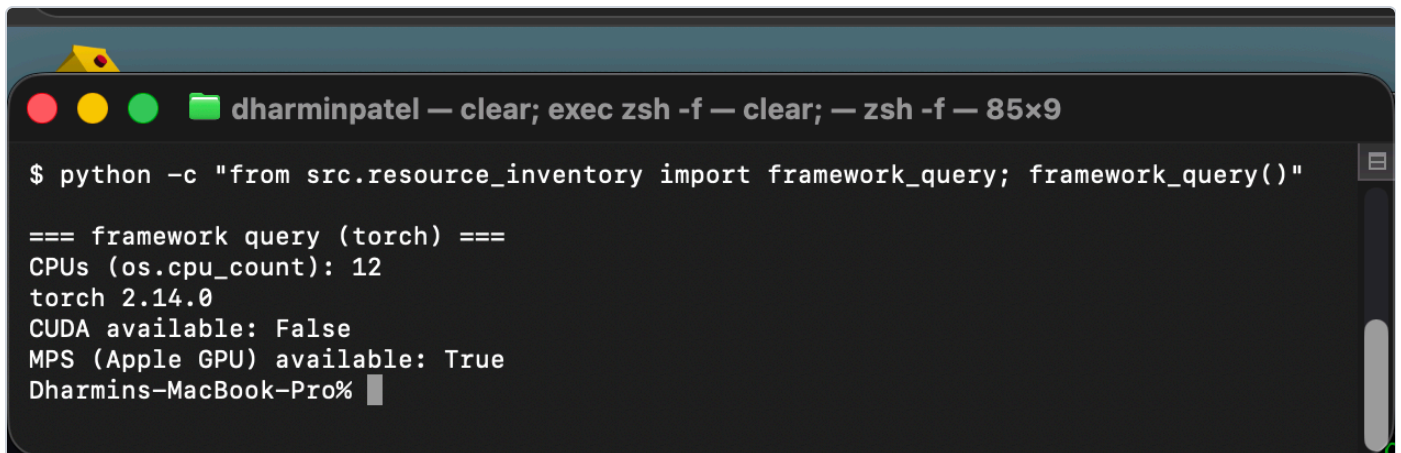
Colab's answer depends on the runtime type selected under **Runtime ► Change runtime type**. On a CPU-only runtime `nvidia-smi` is absent and `torch.cuda.is_available()` returns `False` ; on a GPU runtime `nvidia-smi` reports the attached accelerator and CUDA is available. The runtime used for this evidence is recorded in the screenshot below.



A terminal window titled "dharminpatel — clear; exec zsh -f — clear; — zsh -f — 80x16". The terminal shows the output of the command `system_profiler SPHardwareDataType SPDisplaysDataType` with "(serials removed)". The output lists hardware details for a MacBook Pro, including model name, identifier, chip (Apple M2 Pro), total number of cores (12), memory (16 GB), and Apple M2 Pro specific details like chipset model, GPU type, bus, and display type. The prompt is "Dharmins-MacBook-Pro%".

```
$ system_profiler SPHardwareDataType SPDisplaysDataType (serials removed)
  Model Name: MacBook Pro
  Model Identifier: Mac14,10
  Chip: Apple M2 Pro
  Total Number of Cores: 12 (8 Performance and 4 Efficiency)
  Memory: 16 GB
  Apple M2 Pro:
    Chipset Model: Apple M2 Pro
    Type: GPU
    Bus: Built-In
    Total Number of Cores: 19
    Metal Support: Metal 4
    Display Type: Built-in Liquid Retina XDR Display
    Connection Type: Internal
Dharmins-MacBook-Pro%
```

§3.8 Local system_profiler output, serial cropped



A terminal window titled "dharminpatel — clear; exec zsh -f — clear; — zsh -f — 85x9". The terminal shows the output of the command `python -c "from src.resource_inventory import framework_query; framework_query()"`. The output displays framework query results for torch, showing CPU count (12), torch version (2.14.0), CUDA availability (False), and MPS (Apple GPU) availability (True). The prompt is "Dharmins-MacBook-Pro%".

```
$ python -c "from src.resource_inventory import framework_query; framework_query()"

=== framework query (torch) ===
CPUs (os.cpu_count): 12
torch 2.14.0
CUDA available: False
MPS (Apple GPU) available: True
Dharmins-MacBook-Pro%
```

§3.8 Local torch query showing CUDA False and MPS True

The screenshot shows a Google Colab notebook interface with the following components:

- Table of contents (left sidebar):**
 - IT 7075C — Colab evidence...
 - §3.4 — Mount Drive and ...
 - ... Restart the runtime ...
 - §3.5 — Model call with th...
 - §3.8 — Resource inventor...
 - + Section
- Code cell [12]:**

```
2 import os
3
4 os.environ['ANTHROPIC_API_KEY'] = userdata.get('ANTHROPIC_API_KEY') # key
   never appears in the notebook
5 !pip install -q anthropic
```

Execution progress: 1.3/1.3 MB 21.3 MB/s eta 0:00:00
- Code cell [13]:**

```
1 from anthropic import Anthropic
2
3 client = Anthropic() # reads ANTHROPIC_API_KEY from the environment
4 msg = client.messages.create(
5     model='claude-opus-5', max_tokens=100,
6     messages=[{'role': 'user', 'content': 'Reply with one short sentence
   confirming the API call worked.'}],
7 )
8 print(msg.content[0].text)
```

Output: API call successful – everything's working as expected.
- Section header:** §3.8 — Resource inventory on the Colab runtime
- Code cell [14]:**

```
1 # System utility. If the runtime is CPU-only, nvidia-smi is absent – that is a
   valid result.
2 !nvidia-smi || echo 'no NVIDIA GPU attached (Runtime > Change runtime type to
   request one)'
3 !lscpu | head -20
4 !free -h
```

Output:

```
/bin/bash: line 1: nvidia-smi: command not found
no NVIDIA GPU attached (Runtime > Change runtime type to request one)
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               46 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      2
On-line CPU(s) list:         0,1
Vendor ID:                   GenuineIntel
Model name:                  Intel(R) Xeon(R) CPU @ 2.20GHz
CPU family:                  6
Model:                       79
Thread(s) per core:          2
Core(s) per socket:          1
Socket(s):                   1
Stepping:                    0
BogoMIPS:                    4399.99
Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 cl
Hypervisor vendor:           KVM
Virtualization type:         full
```

§3.8 Colab nvidia-smi / lscpu / free output

The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, the notebook name 'colab_evidence.ipynb', and a 'Share' button. Below the top bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar and a 'Run all' button are also present. On the left side, there is a 'Table of contents' panel with a list of sections: 'IT 7075C — Colab evidence...', '§3.4 — Mount Drive and ...', '— Restart the runtime ...', '§3.5 — Model call with th...', and '§3.8 — Resource inventor...'. The main area of the notebook is divided into two sections. The top section displays system information for an Intel(R) Xeon(R) CPU @ 2.20GHz, including CPU family, model, thread(s) per core, core(s) per socket, socket(s), stepping, bogomips, flags, hypervisor vendor, virtualization type, and l1d/l1i cache. The bottom section shows the output of a Python script that queries the torch framework. The script prints the number of CPUs (2), the torch version (2.11.0+cpu), and whether CUDA is available (False). The output is displayed in a dark-themed interface with a blue cursor icon at the bottom center.

```
Model name: Intel(R) Xeon(R) CPU @ 2.20GHz
CPU family: 6
Model: 79
Thread(s) per core: 2
Core(s) per socket: 1
Socket(s): 1
Stepping: 0
BogoMIPS: 4399.99
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 cl
Hypervisor vendor: KVM
Virtualization type: full
L1d cache: 32 KiB (1 instance)
L1i cache: 32 KiB (1 instance)
Mem: total used free shared buff/cache available
Swap: 0B 0B 0B 0B 0B 0B

[15] ✓ Os
1 # Framework query
2 import os, torch
3 print('CPUs:', os.cpu_count())
4 print('torch', torch.__version__)
5 print('CUDA available:', torch.cuda.is_available())
6 if torch.cuda.is_available():
7     print('CUDA device:', torch.cuda.get_device_name(0))

CPUs: 2
torch 2.11.0+cpu
CUDA available: False
```

§3.8 Colab torch query output

§3.9 — Recorded technical demonstration (12 pts)

Link: <PASTE UNLISTED YOUTUBE / DRIVE LINK HERE> (also on page 1)

Running order used in the recording:

#	Segment	Shows
1	Project open in VS Code Insiders; Extensions view	§3.1
2	Activate venv; <code>python -V</code> , which <code>python</code> ; interpreter and kernel pickers	§3.2
3	Edit a file, commit and push from the Source Control panel; <code>ls -l .env</code> beside a clean <code>git status</code>	§3.3
4	Colab: mount Drive, write file, restart runtime, read it back	§3.4
5	<code>python src/model_test.py</code> locally; the same call in Colab from Secrets	§3.5
6	<code>docker ps</code> , network inspect, <code>curl</code> and <code>ping</code> , Juice Shop in the browser	§3.7
7	<code>resource_inventory.py</code> locally and in Colab	§3.8

Problems encountered and resolutions

1. The virtual environment silently stopped working after the project was moved.

Activating `.venv` succeeded without complaint, but the very next command failed with `command not found: python`. The cause was in `.venv/pyvenv.cfg`, which recorded `command = ... -m venv /Users/dharminpatel/ai-tools-setup/.venv` — the environment had been created at an earlier location, and the project directory had since been moved twice. A venv stores absolute paths in its activation script and interpreter symlinks, so it does not survive being relocated. The fix was to discard and rebuild it in place (`rm -rf .venv && python3.11 -m venv .venv && pip install -r requirements.txt`), which is the correct response rather than a workaround: a virtual environment is disposable by design, and `requirements.txt` exists precisely so it can be reconstructed. This is also the strongest argument for `.venv/` being in `.gitignore` — the directory is not portable, so committing it would ship something broken to any other machine.

2. Choosing an interpreter version: newest is not always usable. The Homebrew default is Python 3.14.7, but building the environment on it fails because PyTorch has not yet published 3.14 wheels, and `torch` is required for the §3.8 framework query. The environment was therefore built deliberately on the Python 3.11 framework build. The lesson is that the constraint flows from the dependencies, not from the interpreter — the isolated environment is what makes holding an older Python for this project harmless to everything else on the machine.

3. Host-to-guest `ping` fails on Docker Desktop for macOS, and it is not a misconfiguration.

`ping 172.28.0.2` from macOS returned 100% packet loss even though the container was healthy and its web service answered normally. Docker on macOS does not run containers on the host kernel; it runs them inside a Linux virtual machine, and `172.28.0.0/16` exists only inside that VM's network namespace. macOS has no route to it, so ICMP has nowhere to go, while TCP still works because Docker's userland proxy forwards the published port across the VM boundary. The diagnosis

was confirmed by pinging the same container successfully from a second container attached to the same bridge. The resolution was to demonstrate ICMP where it is meaningful — on the lab network — and host-to-guest reachability at layer 4 and layer 7 via the published port. Time was lost here treating a platform boundary as a configuration error; checking the platform's architecture first would have been quicker than re-checking the network settings.

One decision that would be made differently

The `.env` file would have been created from `.env.example` on day one, before any code was written. The safe pattern was adopted early enough that no key was ever committed, but the ordering was luck as much as design: the ignore rule and the example file were added alongside the first real credential rather than ahead of it. The correct discipline is to commit `.gitignore` and `.env.example` in the very first commit, so the repository is incapable of accepting a secret before one exists to leak — with `git rm --cached` and a rewritten history being the only remedy afterwards, and a rotated key the only safe assumption. A second, smaller change: the environment would be pinned with `pip freeze > requirements.txt` rather than left as bare package names, so the exact versions in this report are reproducible instead of drifting to whatever is current at the next install.